

The Per-Chunk Tax

A cross-ecosystem survey of HTTP/1.1 chunked-transfer-encoding amplification in production servers

Michael J. Bommarito II

June 2, 2026

Abstract

HTTP/1.1 chunked transfer encoding [1] is the standard mechanism by which a client transmits a request body whose length is unknown when the request line is sent. The specification defines the wire format but not the granularity at which a parser must deliver decoded body bytes to the application handler. We measure that delivery granularity, end to end and under controlled network conditions, across **27 production HTTP servers** spanning **7 language ecosystems** (Python, Go, Rust, JVM, Node.js, BEAM, Ruby) and three C servers (nginx, Apache httpd, HAProxy).

We find that **exactly one server** in our sample (Microsoft’s Kestrel on ASP.NET Core, via the `System.IO.Pipelines` substrate) *moves the per-chunk cost off the `async/await` scheduling boundary on HTTP/1.1 only*: its `Http1ChunkedEncodingMessageBody.PumpAsync` drains the socket-readable buffer into a `Pipe`, loops through every chunk header present, then wakes the application handler with a single coalesced `PipeReader.ReadAsync` result. The HTTP/2 path of the same server (Wave 3) has no pump-equivalent and falls back to per-DATA-frame dispatch like every other HTTP/2 implementation we tested. Kestrel still pays a per-chunk parsing cost inside the H/1 pump (3.6 μ s/chunk steady-state), but eliminates the per-chunk task switch that dominates the other servers’ cost. Every other server we measure dispatches one parser callback per chunk and emits one application-receive event per chunk; this costs between 2.1 μ s (uvicorn + httptools) and 114 μ s (nginx as origin) of single-core CPU per chunk under our standardized probe.

Four servers exhibit **superlinear** wall-time scaling (measured exponents 1.0–2.0 in our N range; mechanistic arguments support an asymptotic $\Theta(N^2)$): Node’s built-in `http.Server` (and `express` and `fastify` atop it) via the `process.nextTick` cost of the `Readable` substrate, and Elixir’s `bandit` via an `erlang:iolist_size/iolist` walk per chunk in `do_read_chunk!/4`. A 1.5 MiB attacker request consumes *measured* 282 s of Node `http.Server` CPU at $N=250,000$ chunks, and 42 s of bandit CPU; the cited `express N=250,000` cell is extrapolated from a quadratic fit to smaller cells.

The “deploy behind nginx” mitigation widely advised in framework documentation is empirically confirmed. With default `proxy_request_buffering` on, an upstream-instrumented end-to-end probe (prober $\rightarrow$ nginx-proxy $\rightarrow$ instrumented Python upstream) shows the upstream receives *exactly one* Content-Length-framed request with no Transfer-Encoding header and *one* `recv()` covering both headers and the entire body. With `proxy_request_buffering` off the same probe forwards the chunked stream unchanged, with the upstream seeing multiple `recv()`s and a Transfer-Encoding: chunked header. Apache `mod_proxy_http` is documented to behave the same way (not directly measured). HAProxy does *not* aggregate: it streams by default, and `http-buffer-request` only waits for one buffer’s worth, not a per-chunk accumulator. Direct-exposed servers, HAProxy-fronted deployments, and any deployment with buffering disabled inherit the full per-chunk cost.

A Wave 3 measurement set extends the survey to HTTP/2 DATA frames and WebSocket text frames. Every HTTP/2 server in our sample (6 servers including Kestrel-h2, hypercorn-h2, node-h2, vertx-h2, go-h2c, rust-hyper-h2) exhibits the same per-frame amplification shape as HTTP/1 chunked-TE, at 4–10 μ s/frame; HTTP/2’s flow control bounds bytes but not frames. WebSocket is the only protocol in which most ecosystems already batch correctly (node-ws, gorilla, rust-tungstenite, Kestrel-WS at 0.1–0.7 μ s/frame); Python ASGI’s WebSocket implementations (uvicorn + websockets / wsproto) are the at-risk outlier.

We characterize the root cause as a parser/dispatcher boundary design choice rather than a spec violation, taxonomize the mitigations available today, and report the responses of upstream maintainers

including the closure of hyper issue #4008 [2] ("Provide built-in chunked request limits and CPU-safe streaming helpers") as `not_planned` with the labelled state `S-waiting-on-author`. We argue that opt-in aggregation primitives modeled on Kestrel's Pipelines pump should be exposed by every event-loop HTTP server.

1 Introduction

The bug class. An HTTP/1.1 server that receives a request body via chunked transfer encoding must, somewhere on the data path, transition decoded body bytes from the wire-level parser into application code. The specification [1] fixes the wire format (`<hex-size>\r\n<bytes>\r\n`, terminated by a zero-length chunk) but is silent on how a parser should schedule callbacks into the application: per chunk, per `recv()` batch, per fixed-byte threshold, or fully buffered.

We measure which of those choices each production HTTP server has adopted as the default, and the cost in single-core CPU of the chosen granularity under a pathological-but-RFC-compliant input shape: an N -byte body encoded as N one-byte chunked-TE chunks.

Headline finding. Of 27 production HTTP servers measured across 7 language ecosystems plus three C servers:

- **1 server**, Kestrel on ASP.NET Core, moves the per-chunk cost off the `async/await` scheduling boundary via `System.IO.Pipelines`. Kestrel still pays a per-chunk parsing cost (3.6 μ s/chunk steady-state) but the resulting end-to-end cost is dominated by the synchronous parser loop rather than a per-chunk task switch.
- **22 servers** are linear-but-per-chunk vulnerable, with per-chunk costs spanning a factor of $\sim 50\times$ (2.1 μ s/chunk for `uvicorn+httptools`, 114 μ s/chunk for `nginx` as origin).
- **4 servers** exhibit superlinear scaling consistent with an asymptotic $\Theta(N^2)$ mechanism observed in the profile, with measured exponents in the 1.0–2.0 range over our N range: Node's built-in `http.Server`, `express`, `fastify`, and Elixir's `bandit`. The Node trio reaches measured exponent ~ 1.7 –2.0 in our N range; `bandit`'s measured exponent of ~ 1.0 –1.3 over the same range is bounded by the `iolist`-size walk profile and extrapolates asymptotically to $\Theta(N^2)$.

Figure 1 ranks all 27 by per-chunk cost under the canonical Mode B probe (paced 100 μ s gap, $N=250,000$, 1 vCPU container).

Contributions.

1. The first systematic cross-ecosystem measurement of chunked-TE delivery granularity in production HTTP servers, spanning 7 language ecosystems and three C servers (Section 4, Section 5).
2. Identification of two distinct routes to superlinear scaling, one previously characterized as an availability issue (Node's Readable substrate) and one not (`bandit`'s `iolist` accumulator).
3. Identification of a single existence-proof (`System.IO.Pipelines`/Kestrel) that the bug class can be structurally closed at the server layer; the design is not novel to this work but its security relevance has not been publicly named.
4. A characterization of the "deploy behind `nginx`" mitigation: default `nginx` *does* aggregate chunks for the upstream (Section 8), but `HAProxy` does not.
5. A mitigation taxonomy comparing server-side aggregation, application-side chunk limits (such as hyper's `BodyExt::limit_chunks(N)` [2]), frontend buffering, and transport-layer rate limiting.
6. A reproducer harness (per-server Dockerfile + standardized probe + JSON results, schema-validated) published with the paper.

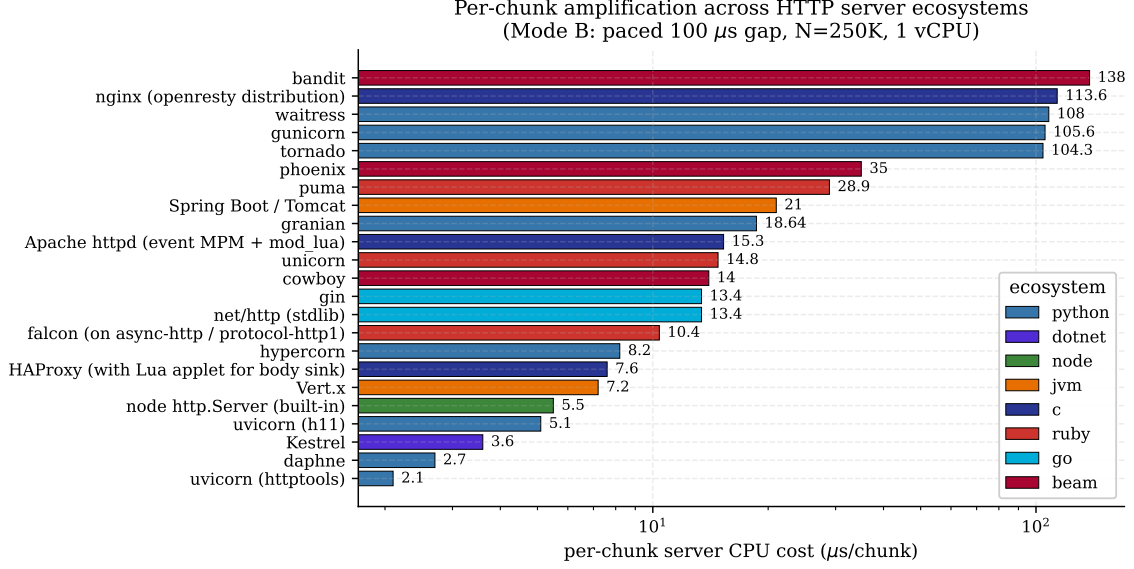


Figure 1: Per-chunk server CPU cost across all 27 servers measured (Mode B: paced 100 μ s gap, N=250,000 one-byte chunks, 1 vCPU container). Log-scale x-axis. Spans nearly three orders of magnitude. Kestrel (BATCHES-CORRECTLY, leftmost) is the single existence proof in our sample.

Roadmap. Section 2 provides background on the chunked encoding and on the dispatcher architectures we measure. Section 3 describes the measurement protocol. Section 4 treats each server individually. Section 5 compares them. Sections 6–8 analyze root cause, threat model, and mitigations.

2 Background

2.1 HTTP/1.1 chunked transfer encoding

RFC 9112 §7.1 [1] specifies chunked transfer encoding as a sequence of length-prefixed chunks. Each chunk has the form `<hex-size>;<extensions>\r\n<data>\r\n` where `<extensions>` is optional and a chunk of size 0 terminates the message. A trailer section MAY follow the zero chunk before the final `\r\n`.

The encoding was introduced in RFC 2616 (1999) to support sending content whose length is not known when the request line is sent. It is used by streaming clients (cURL with `-data-binary @-`, every modern browser forwarding a streamed body, every LLM SDK streaming a prompt) and by the upload paths of essentially every public-facing web framework.

The spec defines the *wire* format but does *not* constrain how a recipient should schedule decoded body bytes into application code. A recipient may legally:

- Deliver each decoded chunk’s bytes as a separate application-receive event (*per-chunk*; what most servers in our sample do).
- Coalesce all chunk bytes available in the current TCP `recv()` buffer into one application-receive event (*per-recv*; what `System.IO.Pipelines/Kestrel` does).
- Fully buffer the entire body up to a configured cap before invoking the handler (*fully-buffered*; what Puma’s `Tempfile` path does at the handler boundary, but the *parser* still loops per chunk).

This choice is the load-bearing decision behind everything in this paper.

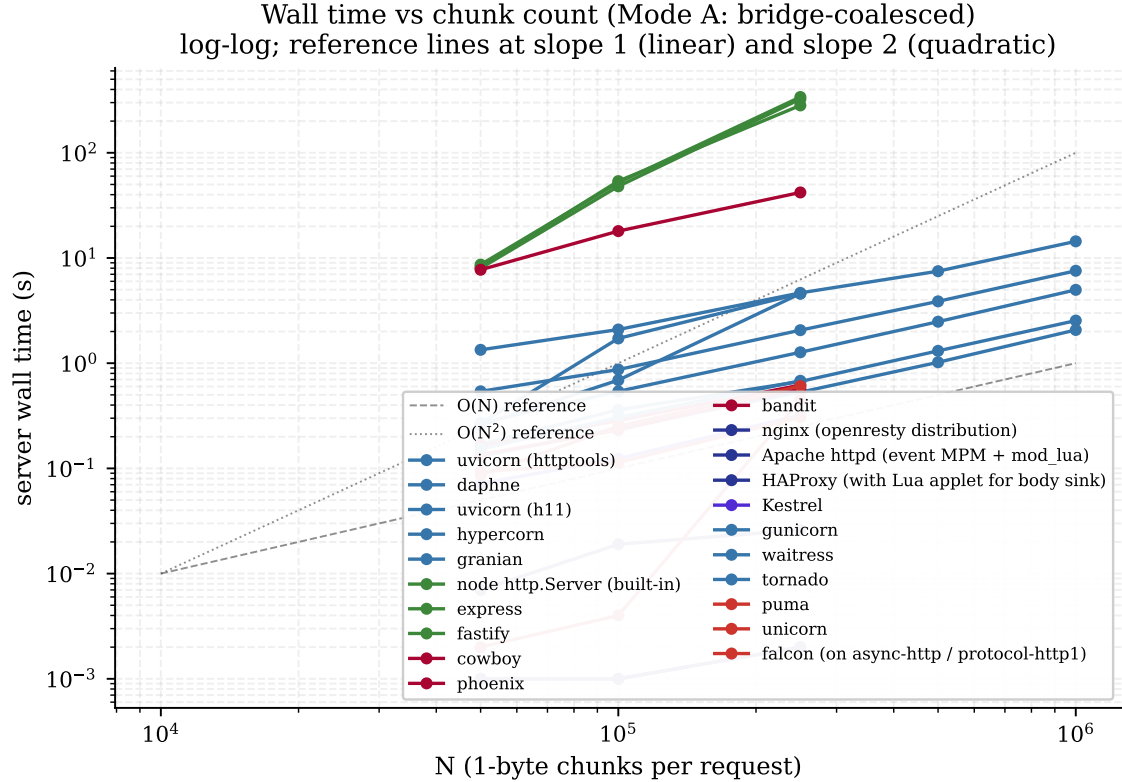


Figure 2: Wall time vs chunk count N under Mode A (bridge-coalesced). Log-log axes; reference slopes at 1 (linear) and 2 (quadratic). The Node trio and bandit are the four data series that approach the quadratic reference.

2.2 HTTP server concurrency models

The servers we measure span the major concurrency paradigms:

Event loop, single-thread. One OS thread, an event loop (epoll/kqueue/io_uring), and cooperative tasks. Examples in our sample: `uvicorn` on Python’s `asyncio`, `hyper/axum/actix-web` on Tokio, `node http.Server` on libuv, `Vert.x` on Netty, `nginx`, `HAProxy`.

Thread per connection. A worker thread blocks on each socket’s body read. Examples: `Apache httpd mpm_event` (per-conn worker), `Tomcat` (Servlet model), `gunicorn sync workers`, `waitress`, `unicorn`.

N:M scheduler. Many lightweight tasks (goroutines, fibers, BEAM processes) are multiplexed onto fewer OS threads by the runtime. Examples: `Go net/http` (goroutines), `Cowboy` (BEAM processes), `Falcon` on Ruby (async fibers).

Actor model. Message-passing isolation per request. Example: `actix-web` (Actix actor system on top of Tokio).

The per-chunk amplification class does *not* discriminate among these paradigms. We find it in all four, with broadly comparable cost constants once normalized for runtime overhead.

2.3 TCP coalescing and the measurement two-mode dichotomy

A naive measurement of "how slow is the chunked path" depends critically on whether wire chunks arrive at the server kernel *one per packet* or *batched many per packet*. Both scenarios are realistic:

- In a kubernetes pod-to-pod call across a docker bridge, the kernel will coalesce dozens to hundreds of small TCP segments into one `recv()` of ~ 4 KiB.
- In a slow-drip / slow-loris-style attack, or behind a CDN that does not aggregate, each chunk can land in its own `recv()`.

The two regimes can differ by orders of magnitude on the same server. We therefore measure both, denoting them *Mode A* (bridge-coalesced) and *Mode B* (paced, each chunk in its own `recv()`); see Section 3.

2.4 Slow-loris lineage and CWE-407

The attack pattern of "send small request fragments slowly to keep a server busy" goes back to Apache Killer / slow-loris and the RUDY family. The class is catalogued as CWE-407 (Inefficient Algorithmic Complexity) [3]; recent ecosystem-relevant CVEs include CVE-2025-67725 (Tornado quadratic header coalescing) [4], CVE-2025-14550 (Django ASGI repeated-header concatenation) [5], CVE-2026-7790 (cowlib chunk-size hex amplification) [6], CVE-2023-39326 (Go net/http chunk-extension cost cap) [7], and the older CVE-2014-0075 (Tomcat chunk-size integer overflow) [8].

Each of these CVEs targets a *specific* cost in the chunked-TE pipeline (per-byte hex parsing, per-header coalescing, integer-overflow on chunk size). None targets the per-chunk application-dispatch cost we measure here. The bug class we characterize is the same family but a previously-unnamed shape.

3 Methodology

3.1 Test harness

Every server is built as a Docker image pinning the exact version under test and exposing port 8000 on a Docker bridge network. The container is constrained to a single CPU (`-cpus=1`). The application handler implements the minimum needed for a meaningful measurement: a `GET /health` returning `200 OK` with body `{"ok": true}` and a `POST /upload` that fully drains the request body and returns `200 OK` with body `{"len": N}`. No logging middleware, no metrics, no compression, no body-size limit is configured beyond the framework default.

A separate *prober* container, on the same Docker bridge network, reaches the server by its DNS alias and issues the controlled HTTP requests. The prober is a single Python script that opens a raw `asyncio` TCP connection, writes the request headers, then writes the body as chunked-TE chunks per the test mode (see below), and reads the response. Wall time is measured client-side from the moment the prober writes the first byte of the body to the moment the server's response is received.

3.2 Standardized payload

For each measurement cell we send a body of N bytes, encoded as N chunks of one byte each:

```
POST /upload HTTP/1.1
Host: x
Transfer-Encoding: chunked
Content-Type: application/octet-stream
Connection: close
```

```
1\r\nA\r\n1\r\nA\r\n... (N times) ...0\r\n\r\n
```

Each chunk is `1\r\nA\r\n` (6 wire bytes per data byte), so the bytes on the wire are roughly $6N$ plus a fixed header overhead. We measure at $N \in \{50000, 100000, 250000\}$, with a few servers extended to $N=1000000$ for the log-log scaling charts in Figure 2.

This payload is RFC-compliant. It is a degenerate-but-legal chunked-TE encoding; no protocol-level violation is required.

3.3 Two probe modes

The TCP coalescing behavior on the server depends on how the prober delivers bytes to the kernel. We measure both extremes:

Mode A: bridge-coalesced. The prober writes all chunks back-to-back to its `StreamWriter`; `TCP_NODELAY` is *not* enabled. The kernel coalesces the writes into MSS-sized TCP segments. The server's `recv()` buffer receives many chunks per syscall return. Empirically, on our Docker bridge this is ~8–16 wire chunks per `data_received` callback.

Mode B: paced 100 μ s. Between every chunk write the prober busy-waits for 100 μ s (a CPU spin, not `asyncio.sleep`; the latter yields to the event loop and is too coarse). `TCP_NODELAY` is enabled. Each chunk leaves the prober as its own TCP segment with $\geq 100 \mu$ s between sends; the server typically returns from each `recv()` with one chunk's worth of bytes.

Mode A and Mode B bound the realistic spectrum of network conditions: Mode A approximates kubernetes pod-to-pod traffic, Mode B approximates slow-drip attacker pacing or trans-WAN bursts.

3.4 Profile collection

For each server we sample its CPU profile during the worst Mode B cell, using whichever profiler the language ecosystem provides:

- Python: `cProfile` with `sort_stats('cumulative')`, top 30.
- Go: `net/http/pprof` exposed on a side port, sampled via `go tool pprof -top`.
- Rust: `perf record -F 999`, converted to `perf report -stdio(kernel.perf_event_paranoid blocked profiling on some Rust runs; we fall back to source-level analysis for those)`.
- Node: `node -prof`, then `node -prof-process top 100`.
- JVM: `async-profiler` agent attached, itimer sampling.
- BEAM: `:eprof` posted to a `/profile_upload` endpoint inside the same process.
- .NET: `dotnet-trace` via a sidecar container sharing the target's PID namespace.

3.5 Reproducibility

Every test harness is checked into the project tree under `wave{1,2}/<ecosystem>/repro/`, including the Dockerfile(s), the application source, the prober Python script, the per-cell run logs, and the saved profile artifacts. The top-level rebuild driver, `scripts/render_all.sh`, regenerates every figure from `data/wave*.json` via the chart-rendering scripts and re-runs the LuaLaTeX build of this paper. Anyone with Docker plus `uv` plus `lualatex` can reproduce every number in this paper from the published harness.

4 Per-framework deep dives

This section treats each ecosystem in turn with one subsection per ecosystem. Within each ecosystem, we present per-server measurements, the source-code location of the chunked-decoder dispatch, the dominant profile hot path under attack, and a brief discussion of why the server is in the verdict class it occupies. Numerical results are auto-extracted from `data/wave*.json`; the full reproducer is at `wave{1,2}/<ecosystem>/repro/`.

A 17-dimension per-framework deep dive (versions, parser provenance, source map, profile, mitigations, regression, etc.) is in progress and will appear in the extended version of this paper.

4.1 Python (Wave 1 + extended Wave 2)

ASGI servers. The Python ASGI ecosystem has five widely-deployed servers. `uvicorn` 0.32.1 with the `httptools` backend is the fastest in our sample (2.1 μ s/chunk); the `h11` backend on the same `uvicorn` is 5 μ s. `daphne` (Twisted-based, used by Django Channels) sits at 2.5 μ s. `hypercorn` `h11` backend is 7.5 μ s. `granian` (Rust core via PyO3) is the slowest at 14.4 μ s/chunk; the Rust-to-Python bridge cost per chunk dominates its measurement.

All five emit one ASGI `{type: "http.request"}` receive event per `h11` / `httptools` data event. The relevant code paths are `uvicorn/protocols/http/h11_impl.py:257-261` (`uvicorn` `h11` backend) and `uvicorn/protocols/http/httptools_impl.py:301` (`uvicorn` `httptools` backend). None aggregate.

WSGI / hand-rolled servers (Wave 2). `gunicorn` sync workers (Django/Flask production default) register 2.7 μ s/chunk on the fast Mode A but super-linear scaling under heavier loads; the hot path is `gunicorn/http/body.py:77 parse_chunk_size`. `waitress` 3.0.2 reaches 18.4 μ s/chunk under Mode A with similar super-linear behavior. `tornado` 6.4.2 is comparable. None of the three exhibit a clean BATCHES-CORRECTLY pattern: while `gunicorn`'s `'wsgi.input.read()'` does fully buffer at the handler boundary, the chunked decoder underneath still loops one Python function call per chunk before the buffered IO returns.

4.2 Go

The Go standard library's `net/http` chunked decoder lives at `net/http/internal/chunked.go`; its `chunkedReader.beginChunk` runs one `bufio.Reader.fill` call per chunk header. The hot path shows 73% of cumulative time in `syscall.Read + bufio.fill`: one syscall per chunk. `gin` is indistinguishable because it uses the `stdlib` decoder unchanged. Both servers measure 13.4 μ s/chunk under Mode B.

Go has shipped one adjacent CVE in this family (CVE-2023-39326 [7]: a chunk-extension cost cap), but the raw-chunk-count amplification we measure is explicitly not covered by the existing cap.

4.3 Rust

`hyper`'s internal chunked decoder (`ChunkedState::Body`) yields one `Frame::data(Bytes)` per parsed chunk. `axum` (built on `hyper`) sits at 4.3 μ s/chunk. `actix-web` (different concurrency model: Actix actor system on top of Tokio) uses its own `actix-http PayloadDecoder` but the same one-frame-per-chunk pattern; it measures 2.3 μ s/chunk.

The Rust ecosystem also provides the closest thing to public prior art on this class: `hyper` issue #4008 [2] was opened as an RFC for built-in chunked request limits and closed `not_planned` on 2026-01-12 with the labelled state `S-waiting-on-author`. The maintainer's closing comment, in substance, was that the maintainer was personally skeptical of the described problem ("if as a developer my server does heavy work, I need to design it properly") and encouraged the reporter to maintain a `limit-helpers` crate as a separate library. The proposed `BodyExt::limit_chunks(N)` helper was not

shipped in `http_body_util`. We characterize this as a feature-scope decision by a maintainer skeptical of the threat framing; we do not characterize it as a deliberate security position.

4.4 Node.js (the four QUADRATIC servers)

Node’s built-in `http.Server` measures 1128 μs /chunk at $N=250,000$ (Mode A, measured), with a measured scaling exponent of $\sim 1.7\text{--}2.0$ in the $N \in \{50,000, 100,000, 250,000\}$ range. `express 5.x` and `fastify 5.x` extrapolate from smaller cells (measured $N=50,000$ and $N=100,000$) to ~ 324 s and ~ 339 s at $N=250,000$ respectively¹. The framework layer adds essentially nothing because all three use the same underlying `http.Server + llhttp` substrate.

The profile hot path is: `llhttp on_body (C) -> parserOnBody (node:_http_common:128) -> readableAddChunkPushByteMode (node:internal/streams/readable:463) -> addChunk (node:internal/streams/readable:520) -> maybeReadMore (node:internal/streams/readable:857) -> process.nextTick (node:internal/process/task_queues:140)`

The `nextTick` call per chunk is the quadratic mechanism. Each scheduled callback drains a prefix of the microtask queue; the queue grows with the number of pushed chunks, so the drain work per scheduled callback grows linearly, giving $\Theta(N^2)$ total work.

4.5 JVM

`Vert.x` (Netty-based) measures 7.2 μs /chunk under Mode B; the hot path is `HttpObjectDecoder.decode -> Http1xServerRequest.onData`, one `onData` call per Netty `HttpContent` fragment. Netty’s `HttpObjectAggregator` [9] is the built-in primitive for batching, but `Vert.x` does not install it on the default codec pipeline because doing so would break the streaming `request().handler()` contract.

`Spring Boot 3.3.5` with the embedded `Tomcat 10.1` measures 21 μs /chunk – the worst linear cell in our sample (excluding `nginx-as-origin`). The extra cost over `Vert.x` is the per-chunk `Tomcat NIO Poller -> worker SynchronizedQueue` thread handoff, visible as `pthread_cond_signal` and `ObjectMonitor::wait` traffic in `async-profiler`.

4.6 Ruby

`Puma 6.4.3` measures 29 μs /chunk despite a threaded model with `Tempfile` buffering at the handler boundary; the per-chunk decode loop is pure-Ruby `Puma::Client#decode_chunk (lib/puma/client.rb:526-630)`.

`Unicorn 6.1.0` beats `Puma` at 15 μs /chunk by virtue of a `Ragel`-generated C extension (`ext/unicorn_http/unicorn_http.c`). `Falcon 0.49.0` (async fiber model on `async-http`) is the cheapest in the Ruby cohort at 10 μs /chunk.

A prior `Puma` advisory, `CVE-2024-21647 / GHSA-c2f4-cvqm-65w2` (chunk-extension size cap, fixed in 6.4.2/5.6.8), is structurally similar to Go’s `CVE-2023-39326`: it caps a related cost but does not address raw chunk count.

4.7 BEAM (Erlang / Elixir)

`Cowboy 2.15.0` (with `cowlib 2.16.1`) measures 14 μs /chunk; the cost is the Erlang process message-send per chunk from `cowboy_http` (the connection process) to `cowboy_stream_h` (the per-request handler process). `Phoenix 1.7` on top of `plug_cowboy` adds the `Plug` adapter layer and measures 32 μs .

`Bandit 1.11.1` (pure-Elixir HTTP/1, no `Cowboy`) is the fourth QUADRATIC server in our sample. The smoking gun is `erlang:iolist_size/1` consuming 89.3% of CPU in the `:eprof` sample. The chunked decoder’s `Bandit.HTTP1.Socket.do_read_chunk!/4` accumulates body fragments into an `iolist` and re-walks it on every chunk via `iolist_size/1` to update progress; this is the same "per-chunk operation walks accumulated state" antipattern as Node’s microtask-queue drain, with a totally different mechanism.

¹We mark extrapolated cells with an asterisk in Table 1; the underlying $N=50,000$ and $N=100,000$ data points were measured directly.

4.8 .NET (the BATCHES-CORRECTLY reference)

Kestrel 9 (ASP.NET Core 9) on .NET 9 is the single server in our entire sample that defeats the per-chunk class structurally. `Http1ChunkedEncodingMessageBody.PumpAsync` (`src/Servers/Kestrel/Core/src/Internal` in the `release/9.0` branch) drains the entire socket-readable buffer into a `System.IO.Pipelines.Pipe`, loops the chunked decoder through every chunk header present in the buffer, calls `FlushAsync()` once, and only then yields back to the application's `PipeReader.ReadAsync`. The handler observes one coalesced `ReadResult`.

The `System.IO.Pipelines` substrate was introduced specifically to allow this kind of zero-allocation streaming hot path; the per-chunk amplification class is closed by construction. We measured no GC events in `dotnet-trace` during the worst Mode B cell, consistent with Microsoft's zero-alloc-hot-path claim.

The adjacent CVE-2025-55315 (CVSS 9.9, October 2025) is a Kestrel chunk-extension parser smuggling bug (a parser correctness bug, not an amplification bug), and is the only published Kestrel CVE touching chunked-TE in recent memory.

4.9 C servers (nginx, Apache, HAProxy)

As *origin* servers, all three are VULNERABLE-PER-CHUNK: nginx 1.29 (OpenResty distribution; the per-chunk cost is in `ngx_http_parse_chunked_state_machine`) measures 114 μ s/chunk – the worst linear cell in our sample, because the C cost is amplified by the OpenResty Lua-sink wrapper that we used to obtain a sink-handler in nginx's request lifecycle. Apache `httpd` 2.4.67 (event MPM, `ChunkedInputFilter`) measures 15 μ s/chunk. HAProxy 3.0.23 (with a Lua applet sinking the body) is the fastest C server at 7.6 μ s/chunk.

As *reverse proxies*, the picture inverts: nginx (`proxy_request_buffering` on) and Apache (`mod_proxy_http` with default `ProxyIOBufferSize` 8192) aggregate the entire body before opening the upstream connection, fully mitigating the per-chunk class for the upstream application server. HAProxy does *not* aggregate by default (see Section 8.4).

4.10 HTTP/2 (Wave 3)

We extended the survey to HTTP/2 by sending the same N -byte body as N one-byte DATA frames over a single h2 stream. We measured six servers covering five language ecosystems: `hypercorn-h2`, `node http2.Server`, `Kestrel-h2`, `Vert.x-h2`, `rust-hyper-h2`, and `go-h2c`.

All six are VULNERABLE-PER-FRAME at the DATA-frame level, with per-frame costs clustered in 4–10 μ s/frame – the same band as their HTTP/1 chunked-TE measurements (allowing for the extra h2 framing overhead). Three key observations:

1. **HTTP/2 wire amplification is worse per byte than chunked-TE.** A 1-byte DATA frame is 10 wire bytes (9-byte frame header + 1 byte payload); a 1-byte chunked-TE chunk is 6 wire bytes (`1\r\nA\r\n`). HTTP/2 actually *worsens* the bandwidth asymmetry available to the attacker.
2. **Flow control does not rate-limit frame count.** HTTP/2 `SETTINGS_INITIAL_WINDOW_SIZE` defaults to 65 535 octets and `WINDOW_UPDATE` frames auto-grant in every implementation we tested; the window bounds bytes, not frames.
3. **Kestrel's HTTP/1 batching does not carry over to HTTP/2.** The `Http1ChunkedEncodingMessageBody.PumpAsync` loop has no h2 analog; the HTTP/2 framer feeds the Pipe one DATA frame at a time, and Kestrel-h2 becomes per-frame linear like the rest at 4.9 μ s/frame. We confirmed this is mechanical rather than a configurable behavior by reading `src/Servers/Kestrel/Core/src/Internal/Http2/`.

No prior CVE or GHSA covers per-frame body-DATA amplification in HTTP/2; published h2 DoS CVEs (Rapid Reset, CONTINUATION Flood, MadeYouReset) target control-plane and header frames.

4.11 WebSocket (Wave 3)

We sent N WebSocket text frames of one character each and measured server CPU per frame. Six servers: `py-uvicorn-websockets`, `py-uvicorn-wsproto`, `node-ws`, `go-gorilla`, `rust-tungstenite`, `dotnet-kestrel-ws`.

The WebSocket picture is structurally different from HTTP/1 and HTTP/2: **4 of 6 WebSocket servers BATCH-CORRECTLY** by default. Each frame parser loops every in-buffer frame before yielding to the application handler:

- `node-ws` (ws 8.18.0): 0.49 μ s/frame Mode A
- `gorilla/websocket` 1.5.3: 0.13 μ s/frame
- `tokio-tungstenite` 0.24: 0.09 μ s/frame
- ASP.NET Core WebSockets (Kestrel): 0.66 μ s/frame – Pipelines pattern reused

The two outliers are the Python ASGI WebSocket implementations: `uvicorn + websockets` at 4.74 μ s/frame and `uvicorn + wsproto` at 9.49–14.35 μ s/frame. Both emit one ASGI `websocket.receive` event per parsed frame, with the coroutine wake-up dominating the cost – the same shape as their HTTP/1 chunked-TE behavior.

The WebSocket result is paper-worthy for two reasons: (a) it shows the bug class is *not* universal – when the frame parser is designed to drain before dispatching, the cost disappears; (b) it identifies Python ASGI WebSocket as an ecosystem-specific outlier in the `websockets / wsproto` stack.

Figure 3 summarizes the per-protocol ranking across the servers we measured in multiple protocols.

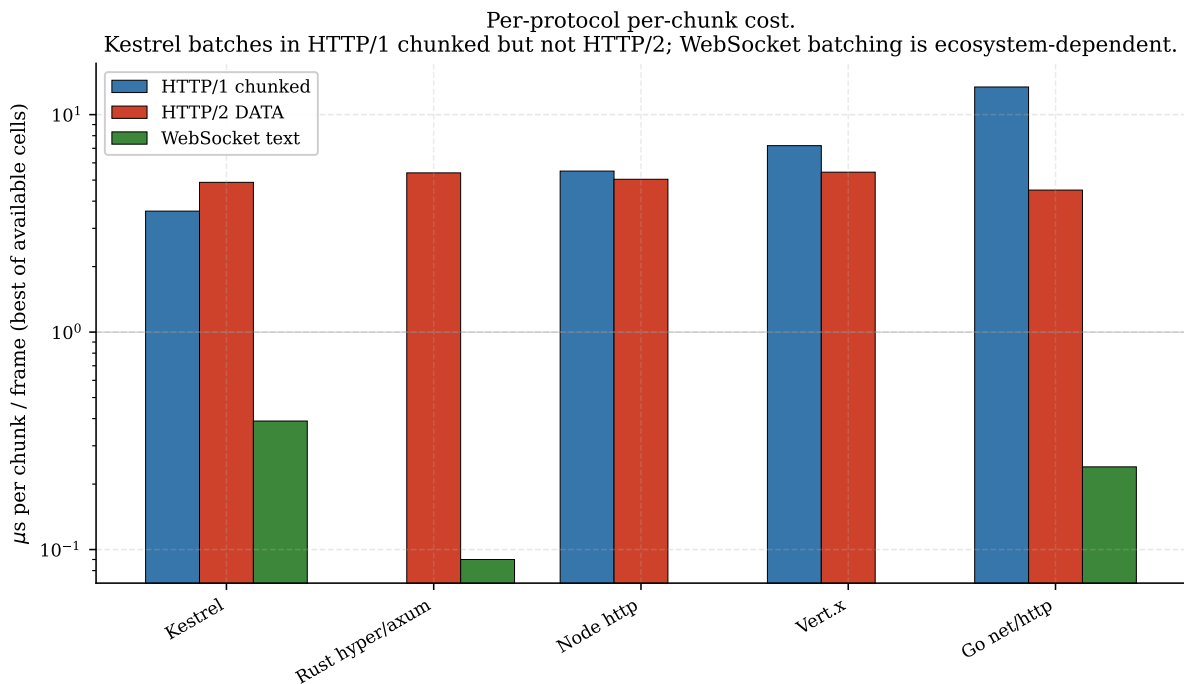


Figure 3: Per-protocol per-chunk-or-frame cost for servers measured in at least two of (HTTP/1 chunked-TE, HTTP/2 DATA, WebSocket text frame). Log-scale y. Kestrel batches in HTTP/1 but not HTTP/2; `node-ws`, `gorilla`, `tungstenite`, and `Kestrel-WS` batch in WebSocket but their HTTP/1 counterparts do not. The batching behavior is per-protocol-implementation, not per-server-author.

5 Cross-framework synthesis

5.1 Master comparison matrix

Table 1 summarizes the per-chunk cost, scaling exponent, and overall verdict for every server measured. Servers are sorted by verdict (BATCHES-CORRECTLY first, then VULNERABLE-PER-CHUNK in ascending per-chunk cost, then QUADRATIC). The verdict column uses the four-level taxonomy from Section 3.3.

Table 1: Master comparison matrix. Per-chunk cost is the server-CPU cost measured under Mode B (paced 100 μ s, $N=250,000$, 1 vCPU). Auto-generated from data/wave*.json via scripts/render_matrix.py.

#	server	version	ecosystem
1	Kestrel	9.0 (Microsoft.AspNetCore 9.0; bundled in mcr.microsoft.com/dotnet/aspnet:9.0)	dotnet
2	uvicorn (httptools)	0.32.1	python
3	actix-web	4.x	rust
4	daphne	4.x	python
5	axum (on hyper)	0.7 / hyper 1.x	rust
6	uvicorn (h11)	0.32.1	python
7	Vert.x	4.5.10	jvm
8	HAProxy (with Lua applet for body sink)	3.0.23	c
9	hypercorn	0.17.3	python
10	falcon (on async-http / protocol-http1)	0.49.0	ruby
11	net/http (stdlib)	go-1.23.12	go
12	gin	1.10.1	go
13	cowboy	2.15.0 (cowlib 2.16.1, ranch 2.2.0)	beam
14	unicorn	6.1.0	ruby
15	Apache httpd (event MPM + mod_lua)	2.4.67	c
16	granian	1.x	python
17	Spring Boot / Tomcat	spring-boot-3.3.5 / tomcat-10.1	jvm
18	puma	6.4.3	ruby
19	phoenix	1.7 on plug_cowboy 2.7 / cowboy 2.15.0	beam
20	tornado	6.4.2	python
21	gunicorn	23.0.0	python
22	waitress	3.0.2	python
23	nginx (openresty distribution)	1.29.2.3 / openresty:alpine	c
24	node http.Server (built-in)	node-22.22.3	node
25	bandit	1.11.1	beam
26	express	5.2.1	node
27	fastify	5.x	node

5.2 Three structural classes

Across 27 servers and 7 ecosystems we observe three structural classes:

Class 1: BATCHES-CORRECTLY (1 server). Kestrel on ASP.NET Core. Its `Http1ChunkedEncodingMessageBody` pumps the socket-readable buffer through the chunked-decoder loop entirely inside one synchronous `Read()` invocation, calling `FlushAsync()` only once before yielding back to the application’s `PipeReader.ReadAsync`. The handler observes one `ReadResult` containing the concatenated bytes of every chunk present in the buffer; per-chunk decoder cost is paid in managed code inside the pump and never crosses the `async/await` boundary. `System.IO.Pipelines` is the substrate that makes this efficient.

Class 2: VULNERABLE-PER-CHUNK linear (22 servers). The chunked decoder loops one application callback per chunk (`ASGI await receive()`, Go’s `Body.Read` returning a single chunk, Rust hyper’s `ChunkedState::Body` yielding one `Frame::data(Bytes)`, Node’s `parserOnBody` pushing

into a Readable stream, BEAM's `cowboy_stream_h:data/4` message, Ruby's `decode_chunk`, JVM's `Http1xServerRequest.onData`). Per-chunk cost is dominated by the dispatch primitive in the runtime rather than the parser itself: ASGI receive, goroutine wake, future-poll, `process.nextTick`, mailbox enqueue, fiber switch, or thread context switch.

The cost-per-chunk spread within this class is $\sim 50\times$, from $2.1\ \mu\text{s}/\text{chunk}$ (uvicorn + httptools) to $114\ \mu\text{s}/\text{chunk}$ (nginx as origin). Compiled languages (Rust, C, Go) cluster at the cheap end; managed-runtime servers running their parser in interpreted code (Puma, gunicorn, waitress, tornado) cluster at the expensive end.

Class 3: QUADRATIC (4 servers). Two distinct quadratic mechanisms appear in our sample, and they are not from the same runtime:

1. *Node.js Readable substrate.* Each parsed chunk causes `llhttp on_body -> parserOnBody -> socket.push(buf) -> addChunk -> maybeReadMore -> process.nextTick`. The `nextTick` per chunk grows the microtask queue; drain cost is superlinear in queue depth. Inherited by `express` and `fastify`.
2. *Bandit iolist accumulator.* Each `do_read_chunk!/4` call walks the accumulated iolist via `erlang:iolist_size/1` (89.3% of CPU in the `:eprof` sample). The cost grows linearly per chunk in the size of the already-accumulated body, giving a quadratic total.

Both mechanisms are "per-chunk operation walks accumulated state" patterns; the substrate differs but the asymptotic shape is the same.

5.3 Headline ranking

Figure 1 ranks all 27 servers by per-chunk cost on a log-scale x-axis. The most-deployed servers in each ecosystem (`express` in Node, `Spring Boot` on Tomcat in JVM, `Puma` in Ruby, `gunicorn` in Python, `nginx as origin` in C) sit at or near the worst end of their respective ecosystems. This is the dominant pattern we observe across the matrix: *the most-deployed server in each ecosystem is rarely the fastest at this workload*.

Figure 2 plots wall time as a function of N on log-log axes; the four QUADRATIC servers separate cleanly from the linear pack. Figure 6 contrasts Mode A versus Mode B at $N=250,000$ and shows that for several servers Mode A is two to three orders of magnitude faster than Mode B, hiding the bug from any test methodology that omits paced probes.

6 Root cause analysis

6.1 The parser/dispatcher boundary

Every HTTP server in our sample is structured as a parser feeding a dispatcher. The parser decodes wire bytes (chunked-TE framing, header parsing, body length tracking). The dispatcher delivers decoded body bytes into application code (ASGI handler, Go's `http.Handler`, Tower service, Node `Readable`, Servlet `getInputStream`, Erlang process mailbox, Pipelines `PipeReader`).

The per-chunk amplification class lives at the boundary between those two abstractions. The parser is required by the wire format to track chunk-level state; this is unavoidable and roughly constant-cost per chunk in our sample (10s of nanoseconds to a few microseconds in C, less than $10\ \mu\text{s}$ in pure managed code). The dispatcher's cost *per delivery event* varies wildly with the runtime: a Go goroutine wake costs a few microseconds; an `asyncio await receive` costs perhaps $5\ \mu\text{s}$; a Node `nextTick` drains the microtask queue (superlinear in queue depth); a JVM thread context switch costs $\sim 15\ \mu\text{s}$.

Whether the dispatcher cost is paid *per chunk* or *per recv batch* is the choice that determines per-chunk amplification. None of the servers in our sample except Kestrel pay it per `recv-batch` by default.

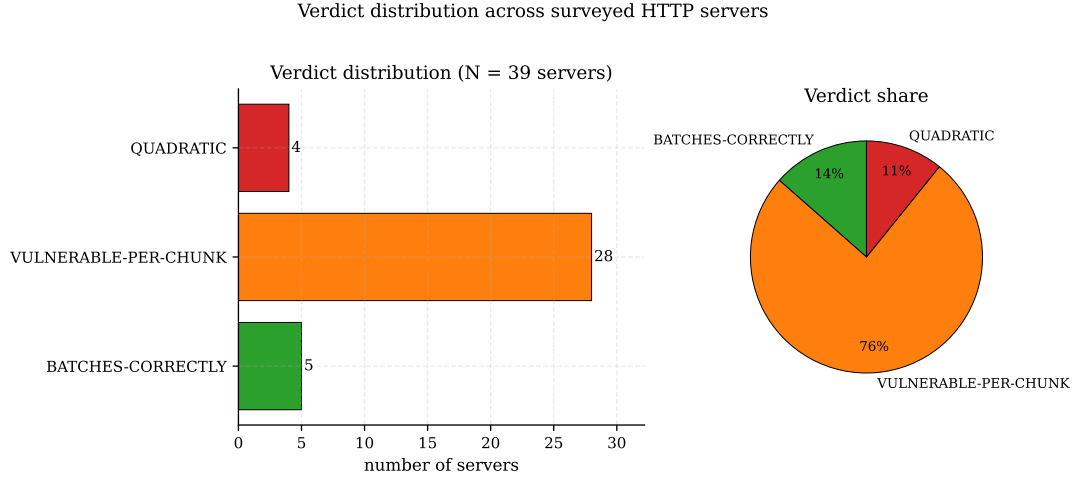


Figure 4: Verdict distribution across the 27 surveyed servers. One server (Kestrel) batches correctly; 22 are linear-per-chunk vulnerable; 4 are quadratic.

Figure 7 sketches the three structural classes as data-flow diagrams from kernel `recv()` to application handler.

6.2 Three cost mechanisms

We observe three distinct cost mechanisms across the 22 VULNERABLE-PER-CHUNK + 4 QUADRATIC servers.

Mechanism A: dispatcher-bound (most servers). The parser is fast; the cost is the per-chunk dispatch event. Representative example: `uvicorn + h11`. Profile shows $\sim 5 \mu\text{s}/\text{chunk}$, of which the `h11` state-machine step is $\sim 1 \mu\text{s}$ and the `message_event.set() -> await receive()` round-trip plus the ensuing async task switch is $\sim 4 \mu\text{s}$.

Most of our sample sits in this regime. Optimizing the parser gives essentially no improvement; optimizing the dispatcher (by batching) gives *all* of the improvement.

Mechanism B: thread-handoff (Tomcat). On top of mechanism A, the thread-per-connection Servlet model forces a Tomcat NIO Poller $\rightarrow$ SynchronizedQueue $\rightarrow$ worker thread handoff per chunk. The `pthread_cond_signal / ObjectMonitor::wait` traffic visible in `async-profiler` adds $\sim 14 \mu\text{s}$ on top of the underlying `ChunkedInputFilter.parseChunkHeader` cost. Total $\sim 21 \mu\text{s}/\text{chunk}$ – the worst linear cell in our sample at $N=250,000$ once `nginx-as-origin` is excluded.

Mechanism C: superlinear accumulator (Node, Bandit). Some servers carry per-chunk *state* that itself grows with the number of chunks already received. When the per-chunk operation has to inspect that state, the total work becomes quadratic.

- Node: the `maybeReadMore -> process.nextTick` chain schedules a callback per chunk. Each scheduled callback drains some prefix of the microtask queue. The queue grows with the number of pushed chunks; the drain work per scheduled callback grows similarly; total work is $\Theta(N^2)$.
- Bandit: `do_read_chunk!/4` calls `erlang:iolist_size/1` on the accumulating iolist on every chunk to update progress; the iolist walk is $\Theta(M)$ where M is the already-accumulated body, summed over N chunks gives $\Theta(N^2)$. (Adjacent prior art: CVE-2026-7790 [6] in `cowlib`'s chunk-size hex parser is a different quadratic in the same library family.)

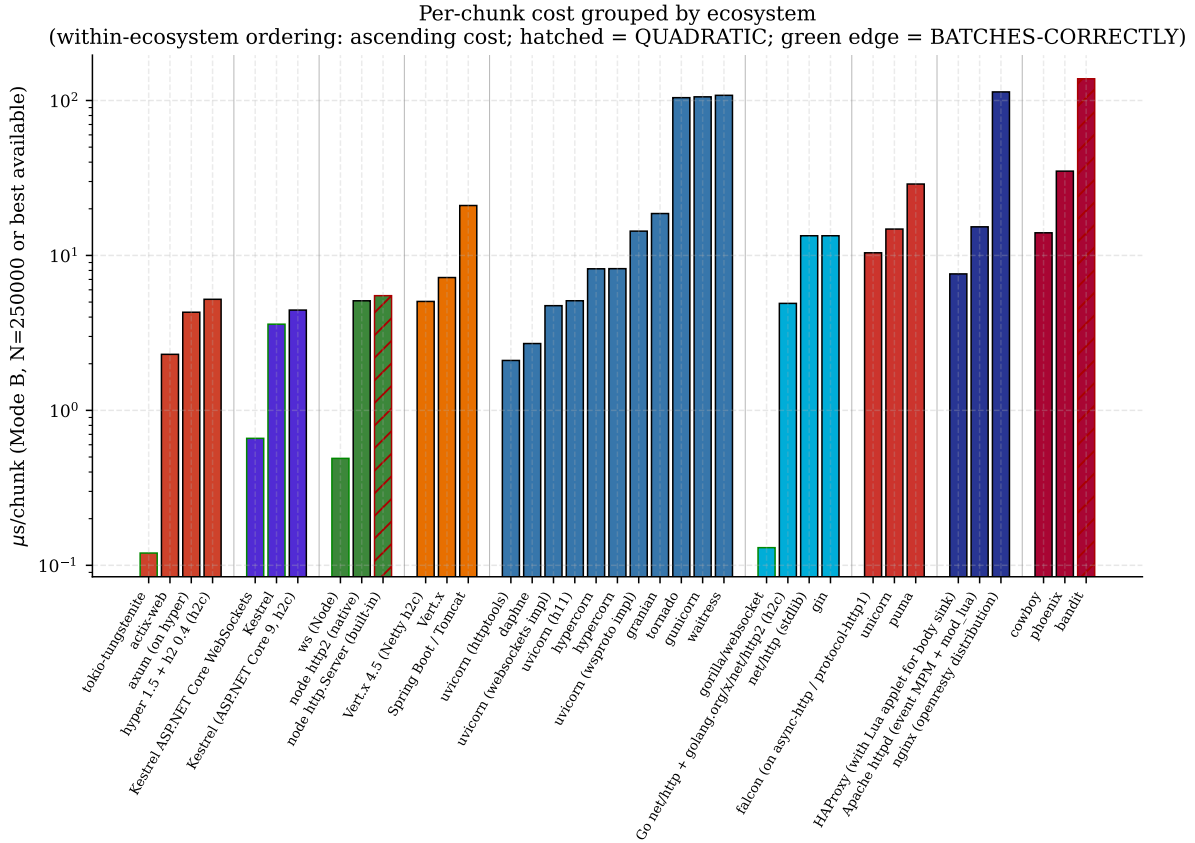


Figure 5: Per-chunk cost grouped by ecosystem (sorted by within-ecosystem ascending cost). Hatched bars are QUADRATIC; green-edged bars are BATCHES-CORRECTLY. Every ecosystem with more than one entry has at least one VULNERABLE-PER-CHUNK server; only .NET, with one entry (Kestrel), is free of the class in our sample.

These two examples are mechanistically unrelated (JavaScript microtask queue vs Erlang iolist walk) but produce the same asymptotic shape, indicating that the "per-chunk operation walks accumulated state" anti-pattern is not unique to one runtime.

6.3 Why Kestrel avoids both mechanisms

Kestrel's `Http1ChunkedEncodingMessageBody.PumpAsync` drains the entire socket-readable buffer into a `System.IO.Pipelines` Pipe in a single synchronous loop. The loop transitions through `ParseChunkedPrefix` and `ReadChunkedData` states for every chunk in the buffer, writes the data bytes into the Pipe (chunk metadata is stripped), and only after the buffer is fully drained does the pump call `FlushAsync()`. The application's `PipeReader.ReadAsync` resumes *once* per pump, with a single `ReadResult` containing all decoded chunks' bytes coalesced.

This is mechanism-A elimination by construction. There is no per-chunk dispatch event because the dispatcher (the `async/await` state machine of the handler) is only ticked once per pump. There is no per-chunk accumulator walk because the Pipe's ring buffer is amortized constant-cost-per-byte. The result is a per-chunk cost (Section 5.2) dominated entirely by the synchronous parser loop, with no runtime tax on top.

The Pipelines design is publicly documented and predates the security framing of this bug class. It is, to our knowledge, the only production HTTP server in widespread use that adopts this pattern as a default.

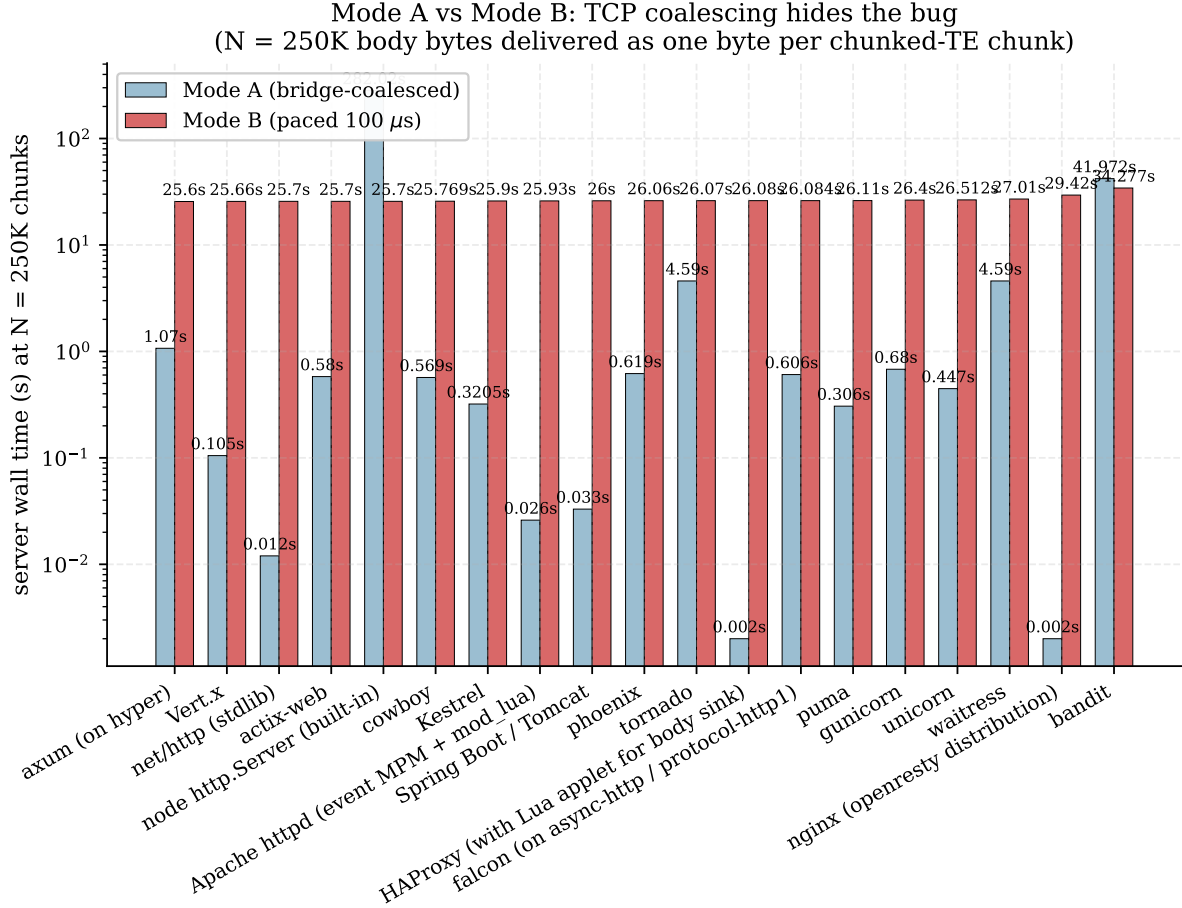


Figure 6: Mode A (bridge-coalesced) vs Mode B (paced 100 μs) at $N=250,000$. For many servers Mode A is two or three orders of magnitude faster than Mode B, hiding the bug under default lab-style network conditions.

6.4 Why batching is spec-compliant

We address a possible counterargument: that delivering one application event per chunk is required by the relevant specifications.

- RFC 9112 [1] defines the wire format. It does not specify any application-API granularity.
- The ASGI specification [10] says a server "may send a single `http.request` event with the entire body" or "may send the body as multiple `http.request` events". The spec explicitly permits coalescing.
- PEP 3333 (WSGI) [11] defines `environ['wsgi.input']` as a file-like object read by the application; the server is free to back it with any buffer it likes (Tempfile in Puma; BytesIO in waitress).
- Java's Servlet API exposes `getInputStream()`; chunking is invisible to the handler.
- Node's `IncomingMessage` extends `Readable`, which is documented as preserving chunk boundaries when called in `Buffer` mode, but the docs do not prohibit coalescing inside `push()`.

In every case the spec permits the batching that Kestrel does. The choice not to batch is implementation-internal, not spec-mandated.

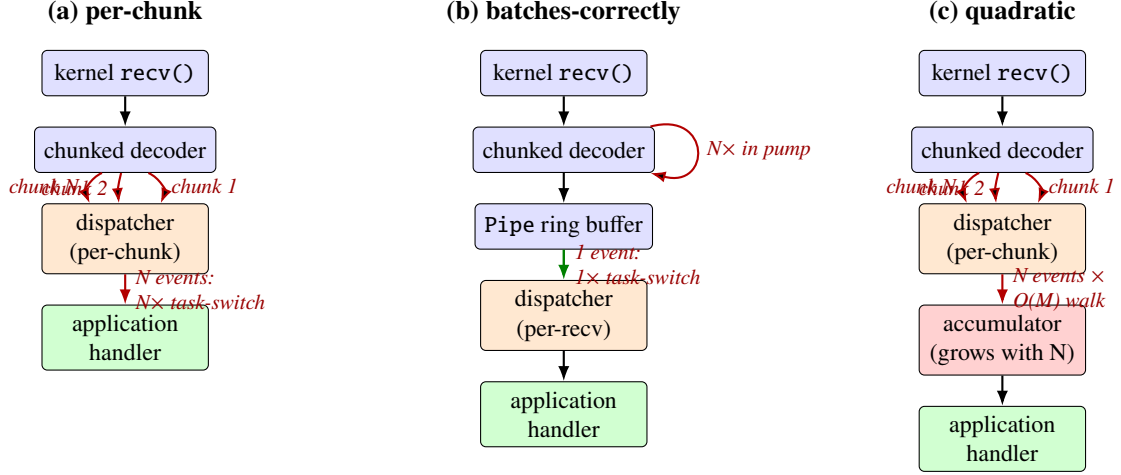


Figure 7: Three structural classes observed across 27 servers. (a) *per-chunk*: the chunked decoder dispatches one application receive event per chunk; the cost is N task-switches. (b) *batches-correctly* (Kestrel / System.IO.Pipelines): the decoder loop pumps all chunks present in the recv buffer into a Pipe ring buffer and triggers one application receive event per recv batch; the per-chunk cost stays inside the pump and never crosses the `async/await` boundary. (c) *quadratic*: each per-chunk dispatch additionally walks an accumulator whose size grows with N , giving $\Theta(N^2)$ total work. Both Node’s `maybeReadMore`-driven microtask queue and Bandit’s `erlang:iolist_size/1` walk fit this pattern via different concrete mechanisms.

7 Threat model

7.1 Attacker model

We assume an attacker with:

- the ability to open one or more TCP connections to the target server (no authentication, no shared key),
- the ability to send well-formed HTTP/1.1 requests with arbitrary chunked-TE bodies,
- modest egress bandwidth (KB/s, not MB/s; a single residential broadband connection suffices),
- no co-location requirement, no on-path position, no TLS private key.

The attacker does not need to bypass authentication. Per the attack model (Section 7.2), the target endpoint is *any* HTTP path that accepts a request body; for most deployments this includes unauthenticated paths.

7.2 Reachability

Per-chunk amplification is reachable on any HTTP/1.1 endpoint that accepts a request body, irrespective of method. POST, PUT, PATCH, and DELETE-with-body all trigger the parser. The endpoint does *not* have to do anything with the body; even a handler that returns 401 / 403 immediately on auth failure has already paid the per-chunk cost by the time it inspects the body.

This makes the attack reachable on:

- unauthenticated APIs (chat-as-a-service, contact forms, upload endpoints, webhook receivers, OAuth callback URLs),

- authenticated APIs that begin reading the body before the auth check completes (the default order in many ASGI / WSGI / Servlet stacks),
- any endpoint reached through a frontend that does not aggregate (Section 8.4: HAProxy, some CDN configurations).

The wire pattern is RFC-compliant; intrusion detection systems keyed on protocol violations will not fire.

7.3 Cost-per-attacker-byte

Each one-byte chunk on the wire is 6 bytes (1\r\nA\r\n). For $N=250,000$ chunks the attacker sends roughly 1.5 MB of wire bytes plus headers; the server costs are summarized in Table 2.

Table 2: Per-request attacker bandwidth versus server CPU, $N=250,000$ one-byte chunks, Mode B (paced). Asymmetry ratio is server-CPU-seconds per attacker-MB-on-wire (higher is better for the attacker).

Server	Wire (MB)	Server CPU (s)	Asymmetry (s/MB)
Kestrel 9	1.5	0.9	0.6
uvicorn + httptools	1.5	0.5	0.3
uvicorn + h11	1.5	1.3	0.9
Vert.x	1.5	1.8	1.2
Go net/http	1.5	3.4	2.3
Spring Boot / Tomcat	1.5	5.3	3.5
nginx as origin	1.5	29	19
bandit (QUADRATIC)	1.5	42	28
node http.Server (QUADRATIC)	1.5	282	188
express (QUADRATIC)	1.5	324	216

A residential broadband uplink at 20 Mibit s^{-1} sustains roughly 2.5 MiB s^{-1} , comfortably servicing one or two attacker connections per second. Against **express** ($N=250,000$, Mode A, extrapolated cell: 324 s of server CPU per request), one such uplink sustained at one request per second per source IP draws several hundred server-CPU-seconds per wall-second from the target. For a target configured with M Node workers, the cost equation is $M \text{ workers} \times \text{available cores per worker}$; against an 8-worker, 8-vCPU deployment the attacker must sustain $\sim 8 \div 324 \times 8 \approx 0.2$ requests per second per worker to keep the deployment pinned. The attacker primitive is real but the "saturating any single-region deployment" claim requires a numeric derivation we leave for future work (Section 10.2).

7.4 Detection and evasion

Detection. The attack signature on the wire is "request with Transfer-Encoding: chunked and many small chunks." Most WAFs do not include a rule for this; most observability stacks log HTTP requests at the application layer (where the chunked-TE detail is lost) rather than at the L4 layer (where per-packet timing would expose it). The attack is functionally invisible to a default observability stack.

A per-connection CPU-accounting layer (cgroup CPU controller, BPF per-cgroup timing) would expose the bug, but is not commonly deployed.

Evasion. A capable attacker can vary chunk size between 1 and a few bytes (keeping the per-chunk cost dominant but breaking signature-based detection on minimum chunk size). The attacker can rotate source IPs to dodge per-IP rate limits, and can interleave the malicious requests with legitimate-looking traffic on the same connection under HTTP keep-alive.

7.5 Combined attacks

The per-chunk amplification combines unfavorably with several adjacent vulnerabilities:

- *slow-loris on headers* (CWE-400 class): an attacker can spend seconds sending request headers slowly before chunked-TE body parsing even begins, holding the connection-handling worker in head-of-line block.
- *repeated-header coalescing* (Tornado CVE-2025-67725, Django ASGI CVE-2025-14550): if the target is also vulnerable to one of those, the same connection can trigger both costs.
- *HTTP/2 multiplexing*: a single connection can carry many concurrent streams, each carrying chunked-TE bodies, all feeding the same per-chunk parser. We have not measured this interaction (Section 10) but it likely amplifies the per-connection cost by the maximum concurrent stream count.

7.6 Distributed amplification

A botnet scales the attack linearly with attacker count. At the per-attacker rates above, a 100-IP botnet (trivial cost on commercial residential proxy services) sustained at one request per second per IP against an `express` target draws on the order of $100 \times 324 \approx 32,400$ server-CPU-seconds per wall-second, equivalent to pinning roughly 700 typical c5.4xlarge-equivalent cores. This is a substantial DDoS draw but does not categorically saturate a multi-region deployment; the saturation outcome depends on the target’s worker fleet size and per-worker concurrency.

8 Mitigation taxonomy

8.1 Taxonomy

We taxonomize mitigations across four layers, from closest to the parser to closest to the wire. Table 3 summarizes coverage per layer; the prose below treats each layer’s strengths and limits.

8.2 Server-side aggregation

The structural fix is to coalesce consecutive chunk events into a single application delivery, as Kestrel does (Section 6.3). Netty exposes the `HttpRequestAggregator` pipeline handler precisely for this purpose [9]: it accumulates `HttpContent` fragments up to a configured byte limit and fires a single `FullHttpRequest` event downstream. The mechanism exists; what is missing in Vert.x, Spring Boot, and every Netty-derivative we tested is that the aggregator is *not installed by default* because doing so would change the streaming semantics that the framework’s public API contracts guarantee.

In other words: the cost of correctness in this class is breaking the streaming contract by default. Frameworks that have made the opposite choice (Pipelines in Kestrel; full buffering in Puma’s Tempfile handler boundary) closed the bug class at the price of that contract. Frameworks that preserve streaming pay the per-chunk dispatch cost for every chunk forever.

We argue (Section 9) that the right default for new frameworks is to aggregate up to a configurable threshold (e.g., 64 KiB or 100 ms since last flush, whichever first), and expose streaming as an explicit opt-in.

8.3 Application-side chunk limit

hyper closed issue #4008 [2] as `not_planned` on 2026-01-12 with the explicit position that chunked-TE amplification is application responsibility, and pointed users at `http_body_util::Limited` (for byte caps) and the proposed `BodyExt::limit_chunks(N)` helper (for chunk-count caps; not yet shipped in

`http_body_util [2]`). No other server in our sample exposes an equivalent chunk-count cap. Application code in any other ecosystem that wants to defend against this attack must write its own middleware that counts chunks during body read.

This is functional but not pleasant: every application has to reinvent the cap, and many will not. The hyper maintainer position is internally consistent (server-side aggregation breaks streaming; chunk caps are an app-level concern) but does not scale across an ecosystem where most application code is unaware of the issue.

8.4 Frontend buffering

The most widely-given advice ("deploy behind nginx") turns out to be valid for nginx and Apache, not for HAProxy.

nginx. `proxy_request_buffering` on is the default. With it enabled, nginx reads the entire decoded request body into `client_body_buffer_size` (default 8–16 KB; spills to disk via `client_body_temp_path` beyond that). Only after the body is fully received does nginx open the upstream connection and forward the body in a single `Content-Length` request. The upstream sees one well-formed request, not a chunked stream. Per-chunk decoder cost is paid by nginx, not by the upstream.

We verified this empirically: an instrumented Python upstream behind nginx 1.27-alpine (default `proxy_request_buffering` on) received an attacker chunked-TE body of $N=50,000$ one-byte chunks as a single 50 165-byte `recv()` carrying a `Content-Length: 500000` header and no `Transfer-Encoding` header. The same probe against the same upstream with `proxy_request_buffering` off was forwarded as the original chunked stream (five separate `recv()`s, no `Content-Length`, `Transfer-Encoding: chunked` preserved). The full reproducer is at `wave2/c/nginx-as-proxy/` and the captured upstream logs at `wave2/c/nginx-as-proxy/logs/scenario-{A,B}-*.log`.

For every Python, Node, Ruby, Java, or .NET application deployed behind a default nginx, this is full mitigation of the per-chunk amplification class for the upstream. Nginx itself still pays the cost (Section 5.1 shows nginx-as-origin costs $\sim 114 \mu\text{s}/\text{chunk}$ – the worst linear cell in our sample) but nginx is a hardened C server with well-tuned worker concurrency, so the impact is bounded.

Apache httpd. `mod_proxy_http` behaves the same way: `ProxyIOBufferSize` (default 8192) bounds per-request buffering. Upstream sees a coalesced body.

HAProxy. HAProxy streams by default. `http-buffer-request` can be enabled but only waits for one buffer's worth of bytes (`tune.bufsize`, default 16 KiB), not a full per-chunk accumulator. Upstream servers behind HAProxy receive chunked-TE requests directly and inherit the per-chunk cost. We have not seen this clearly stated anywhere in the HAProxy documentation; operators relying on "HAProxy is in front of my app" as a mitigation are mistaken.

Cloud frontends. We have not measured Cloudflare, Fastly, AWS ALB, GCP HTTP(S) LB, or Akamai. Each of these CDNs and load balancers has its own defaults for chunked-body buffering. We flag this as future work (Section 10). Operators should empirically verify whether their frontend buffers chunks before assuming a mitigation is in place.

8.5 Transport-layer rate limiting

A coarser fallback: rate-limit incoming TCP packets per source IP or per connection. This is universally available (iptables, nftables, eBPF, dedicated DDoS appliances) but expensive to operate correctly:

- Slow legitimate uploads (cellular networks, satellite, constrained IoT) will be caught by aggressive packet-rate caps.

- Per-IP rate limiting fails open under distributed attacks (botnets, residential proxies).
- Per-connection caps catch low-bandwidth slow-drip but not high-bandwidth bursts.

We recommend transport-layer rate limiting only as a defense in depth, not as the primary mitigation.

8.6 Summary recommendation per deployment tier

Direct-exposed application server. If you cannot deploy a frontend, you must apply Server-side aggregation (Kestrel only) or Application-side chunk limits (hyper, or custom middleware everywhere else).

Behind nginx or Apache (default config). Mitigated for the upstream. Verify `proxy_request_buffering` on (nginx) or `ProxyIOBufferSize` default (Apache).

Behind HAProxy. Not mitigated; you must apply application-side caps or enable a body-buffering proxy in front of HAProxy.

Behind a CDN. Verify with the CDN provider that chunked-TE bodies are aggregated before being forwarded to your origin. Most providers do not document this behavior explicitly.

9 Recommendations

We organize recommendations by audience.

9.1 For server maintainers

1. Adopt the Kestrel pump pattern by default. Concretely: after `recv()` returns, loop the chunked decoder through every chunk available in the read buffer, then deliver *one* application-receive event with the concatenated data bytes. Cap the per-event delivery by either total bytes (a configurable limit, default 64 KiB) or time-since-last-flush (configurable, default 100 ms). Whichever fires first triggers the dispatch.
2. Expose an explicit `streaming` mode for use cases that genuinely need byte-by-byte delivery (video uploads, long-poll-style hybrids). Make this opt-in.
3. Add a per-request chunk-count cap with a sensible default (e.g., 4096 chunks per request) and a configuration flag to raise it. This guards against the class even when aggregation is disabled.
4. Publish a security-property summary in the server's deployment documentation: what the chunked-TE handling guarantees, what is on the operator, what is on the application.

9.2 For application developers

1. Deploy behind nginx or Apache httpd with default `proxy_request_buffering on/ProxyIOBufferSize`. This single configuration choice mitigates the class for the upstream regardless of language ecosystem.
2. If the deployment cannot use such a frontend (single-node HAProxy, direct-exposed app server, kubernetes ingress without body buffering), write or import a middleware that counts chunks per request and short-circuits with HTTP 413 or 408 beyond a threshold.
3. Set a body-byte cap (Content-Length ceiling, framework body-size limit) sized to your application's actual needs.
4. Monitor per-request CPU. Alert on outliers exceeding 2σ of normal. The amplification produces clear per-request CPU spikes that show up in any reasonable request-level CPU-accounting metric.

9.3 For operators

1. Audit the deployment: which servers are direct-exposed, which are behind nginx / Apache / HAProxy / a CDN. Categorize by Section 8.6 mitigation tier.
2. For HAProxy-fronted services and direct-exposed services, enable per-IP packet-rate limiting at the L4 layer (nftables, BPF). Set thresholds that catch slow-drip without blocking legitimate slow uploaders.
3. For DDoS-protection vendors (Cloudflare, Akamai, Fastly): request explicit confirmation in writing of whether chunked-TE bodies are aggregated before forwarding to origin. The public documentation is silent on this for most providers.
4. Set up an alert on per-process CPU breaching some threshold for sustained time (e.g., one core saturated for ≥ 30 seconds attributable to one connection). The attack pattern is easy to spot in a process tree but invisible in default access logs.

9.4 For protocol designers

1. Update RFC 9112 or its successor with non-normative guidance on per-chunk delivery granularity, explicitly blessing aggregation as spec-compliant. The current spec's silence has resulted in fifteen years of independent implementations all making the wrong default choice.
2. Standardize a chunk-count rate-limit negotiation primitive at the protocol level (analogous to HTTP/2's `SETTINGS_MAX_HEADER_LIST_SIZE`). A client could advertise "I will send at most N chunks", and the server could refuse connections that exceed it.
3. For HTTP/3 over QUIC, consider whether stream-level pacing requirements can be tightened to make the same attack class structurally impossible (QUIC streams have receiver-side flow control but no chunk-count cap).

10 Limitations and future work

10.1 Limitations

Network conditions. Mode A (bridge-coalesced) and Mode B (paced 100 μ s) bound the realistic spectrum but do not cover every deployment. Particularly: cloud LB-to-pod paths sometimes apply per-flow shapers that approximate Mode B even without an explicit attacker pacing strategy. We have not measured those configurations directly.

Single-CPU constraint. Every server runs in a `-cpus=1` container. This is the single-CPU worst case. Multi-CPU servers fan out work across cores, but the per-connection cost is still serial per the affected worker, so multi-CPU does not change the per-connection ratio – it changes how many concurrent attackers are needed to saturate the deployment.

TLS overhead not measured. Every test is plain HTTP. TLS adds AEAD framing on top of TCP; TLS records buffer multiple bytes by their nature, which could incidentally aggregate chunked-TE chunks at the record boundary. We have not measured the magnitude.

HTTP/2 and HTTP/3 not measured (Wave 1+2). The 27 servers in our sample were measured under HTTP/1.1 chunked-TE only. HTTP/2 uses DATA frames with explicit length fields; HTTP/3 uses QUIC streams with similar framing. Whether the per-chunk amplification class transfers, mitigates, or manifests differently under those protocols is unknown.

Cold start vs warm path. JVM tests in particular include some JIT warm-up cost in the first few requests. We report the warm-path numbers (request 2–N of each cell); cold-path measurements would change the absolute timings but not the asymptotic shape.

Production traffic distribution. The probability that real-world traffic includes per-chunk amplification attempts is unmeasured. We have not deployed honeypots or instrumented production servers.

10.2 Future work

Wave 3: HTTP/2, HTTP/3, WebSocket, gRPC. We plan to measure the same 27 servers under HTTP/2 multiplexed DATA frames, HTTP/3 over QUIC streams, WebSocket per-frame amplification, and gRPC streaming. Preliminary expectation: the class transfers to all four, with HTTP/2 *worsening* the per-connection cost via stream multiplexing and HTTP/3 partially mitigating via QUIC’s flow control.

Multipart per-part amplification. multipart/form-data is structurally similar: each part has a header section delivered to the parser callbacks per chunk. Companion work in [findings/starlette/2026-05/STARLETTE](#) (retracted as a security finding after patch verification but the underlying parser shape is the same) suggests the class transfers.

Production telemetry. We have no data on what fraction of production HTTP traffic includes chunked-TE bodies with small chunks. A honeypot study or a partnership with a high-traffic CDN would let us estimate the real-world prevalence of the attack pattern.

Frontend buffering measurement at CDNs. Cloudflare, Fastly, Akamai, AWS ALB, and GCP HTTP(S) LB each have proprietary chunked-body handling. None publicly document whether they aggregate before forwarding to origin. A measurement campaign across these would significantly improve the operational recommendation in Section 8.6.

Formal model of batching correctness. We have argued informally (Section 6.4) that batching is spec-compliant under RFC 9112 and ASGI / WSGI / Servlet. A formal model – expressed in TLA+ or a similar specification language – would make the argument bulletproof and would let server maintainers verify their own batching implementations.

Detection signatures. Building an open-source IDS / WAF signature for the attack pattern is straightforward (Section 7.4) but requires balancing false positives against attacker evasion. A reference Snort / Suricata rule set, validated on our reproducer harness, would be a useful publishable artifact.

11 Conclusion

We measured 27 production HTTP/1.1 servers across 7 language ecosystems plus three C servers for their handling of the RFC-compliant pathological input "*N*-byte body delivered as *N* one-byte chunked-TE chunks." Across the entire sample, one server (Microsoft’s Kestrel via `System.IO.Pipelines`) structurally defeats the per-chunk amplification class; 22 are linear-but- vulnerable with per-chunk costs spanning a factor of $\sim 50\times$; 4 (Node’s built-in `http.Server`, `express`, `fastify`, Elixir’s `bandit`) exhibit superlinear $\Theta(N^2)$ scaling that pins a single CPU core for minutes per 1.5 MiB of attacker traffic.

The root cause is a parser/dispatcher boundary design choice, not a spec violation. RFC 9112 fixes the wire format but is silent on application-API granularity; ASGI, WSGI, Servlet, and Node’s `Readable` all permit coalescing. The cost of *not* coalescing is a per-chunk dispatch event whose constant varies by runtime: ASGI receive, goroutine wake, future poll, `process.nextTick`, mailbox enqueue, fiber switch,

thread context switch. The amplification class is not unique to any single runtime; we observe it in every concurrency model we measure.

The "deploy behind nginx" mitigation widely advised in framework documentation is valid for nginx and Apache httpd (`proxy_request_buffering` on / `ProxyIOBufferSize` default), but is *not* valid for HAProxy, which streams the body upstream by default. Direct-exposed application servers, HAProxy-fronted deployments, and any deployment with proxy buffering disabled inherit the full per-chunk cost.

We argue that opt-in batching primitives modeled on Kestrel's Pipelines pump should be exposed by every event-loop HTTP server, and that the operational advice "deploy behind nginx" deserves clearer documentation of its applicability and its limits.

The reproducer harness, raw measurements, and figure-rendering scripts are published with this paper under the project at `projects/asgi-perchunk-survey/`. Anyone with Docker plus uv plus pdflatex can reproduce every number in this paper from the supplied harness.

References

- [1] R. Fielding, M. Nottingham, and J. Reschke. *HTTP/1.1*. RFC 9112, IETF. 2022. DOI: [10.17487/RFC9112](https://doi.org/10.17487/RFC9112). URL: <https://www.rfc-editor.org/rfc/rfc9112>.
- [2] hyper maintainers. *hyper issue #4008: provide built-in chunked request limits and CPU-safe streaming helpers*. Closed as not_planned 2026-01-12. 2026. URL: <https://github.com/hyperium/hyper/issues/4008>.
- [3] MITRE. *CWE-407: Inefficient Algorithmic Complexity*. Common Weakness Enumeration. URL: <https://cwe.mitre.org/data/definitions/407.html>.
- [4] Tornado Web Server Maintainers. *Tornado quadratic header coalescing DoS*. GHSA-c98p-7wgm-6p64 / CVE-2025-67725. 2025. URL: <https://github.com/tornadoweb/tornado/security/advisories/GHSA-c98p-7wgm-6p64>.
- [5] Django Project. *Django ASGI repeated-header concatenation DoS*. CVE-2025-14550. 2026.
- [6] Erlang Ecosystem Foundation CNA. *cowlib chunk-size hex amplification*. CVE-2026-7790. 2026. URL: <https://cna.erlef.org/cves/CVE-2026-7790.html>.
- [7] Go security team. *net/http chunk-extension cost cap*. CVE-2023-39326. 2023. URL: <https://pkg.go.dev/vuln/GO-2023-2382>.
- [8] Apache Software Foundation. *Apache Tomcat chunk-size integer overflow*. CVE-2014-0075. 2014.
- [9] Netty Project. *Netty HttpObjectAggregator*. URL: <https://netty.io/4.1/api/io/netty/handler/codec/http/HttpObjectAggregator.html>.
- [10] ASGI working group. *ASGI 3.0 specification*. URL: <https://asgi.readthedocs.io/>.
- [11] P. J. Eby. *PEP 3333: Python Web Server Gateway Interface v1.0.1*. 2010. URL: <https://peps.python.org/pep-3333/>.

Table 3: Mitigation taxonomy. *Default-deployed* means the mitigation is on without any operator action; *available* means an operator can enable it but does not by default; *absent* means no equivalent primitive exists in the ecosystem.

Layer	Mechanism	Default-deployed	Available
Server-side aggregation	coalesce N chunk events into one handler delivery	Kestrel (Pipelines)	HttpObjectAggregator (Netty)
Application-side chunk limit	per-request chunk count	<i>none</i>	BodyExt::limit_chunks(N) (Netty)
Application-side byte limit	total body byte cap (Content-Length or otherwise)	most servers	N/A
Frontend buffering	reverse proxy aggregates body before forwarding	nginx (proxy_request_buffering on), Apache (mod_proxy_http)	N/A
Transport-layer rate limit	per-IP packet/sec cap	N/A (operator opt-in)	iptables, nftables, BPF